Drop, Reflect and Reuse a Quantum Temporary

ANONYMOUS AUTHOR(S)

A quantum program often allocates a temporary register, uses it, and drops it. Compilers usually reclaim its qubits by uncomputing it, running backwards the gates that acted on it. This can repeat much of the work even when the drop was already safe. We specify a safe drop by what can be observed: just before the drop, no context can tell the program from one that prepared the temporary without the input. This holds exactly when the temporary ends in one fixed pure state at every input, and then at most one reflection on it alone restores its starting state. The test extends to programs that measure, one record of outcomes at a time, and to borrowed registers. A prototype compiler to OpenQASM 3 computes the gate and reuses the freed wire.

1 Specifying a safe drop

Programming with quantum data in the pure setting has a mature treatment in programming languages, from categorical semantics to typed and circuit languages [1, 22, 25, 26]. Recent work adds measurement to that setting as a computational effect [6, 12, 27]. Discarding a register is one half of what measurement needs, and allocating one is the other. Unlike in the classical setting, discarding is not free. Copying classical data and discarding the copy does nothing, but on quantum data it is a measurement [12, §3.4.1], and discarding a register can leave the rest mixed [12, §2.1.2]. This work is about the allocation and deallocation of registers holding quantum data. Consider two programs on a qubit `v`:

```
prog a (v : [2]) : [2] {          prog b (v : [2]) : [2] {
  new t : [2] @ |0>;                 new t : [2] @ |0>;
  H t;                               H t; CX t v;
  drop t; return v }                 drop t; return v }
```

Here `new t : [2] @ |0>` allocates a qubit `t`, of dimension 2, in $|0\rangle$, and `drop t` ends its scope, hiding `t` without a gate. In program (a), the Hadamard gate H puts `t` in the plus state $|+\rangle$, an equal superposition of $|0\rangle$ and $|1\rangle$, whatever `v` held, and another H would restore $|0\rangle$ without touching `v`. Program (b) adds `CX t v`, which flips `v` when `t` is 1. On input $|0\rangle$ it leaves `t` and `v` in a Bell state, an equal superposition of $|00\rangle$ and $|11\rangle$ that no two separate operations can prepare, so dropping `t` leaves `v` mixed. In both programs, `t`'s lifetime ends at the drop, so scope checks accept both. Since gates are unitary and can therefore be undone, compilers usually reclaim a temporary by uncomputing it, running the computation that acted on it backwards [3, pp. 525–526][4, 5, 20], which can take as many gates as the computation itself [17, §1]. Silq and Unqomp automate this by letting only `qfree` gates, which create no superposition, produce the temporary [5, §4.4][20, §2.2], so both refuse program (a), whose one H is not `qfree` [20, §6.2]. We ask when one unitary on the dropped registers alone restores their allocation state, making the drop free.

Now let p allocate `t` in a pure state ψ, run measurement-free code M on `t` and the input `v`, and drop `t`. Its independent witness $\mathcal{S}(R,U)$ prepares `t` by one gate R on `t` alone, while a gate U acts on `v` and fresh registers for M's other temporaries, left implicit below:

$$p(v) = \texttt{new t : [d] @ ψ; M; drop t; return v},$$

$$\mathcal{S}(R,U)(v) = \texttt{new t : [d] @ ψ; R t; U v; drop t; return v}.$$

We call the drop *safe* if $p \equiv \mathcal{S}(R,U)$ for some R and U: no closed context can tell the two programs apart just before the drop, with every register still visible. One pair serves every input: $\exists R,U\ \forall x$, as in RapunSL's $\exists P\ \forall|\psi\rangle$ for its leftover assertion [18, App. B.3]. Registers are finite-dimensional over $\mathbb{C}$, and every unitary is a gate.

2 Checking the equivalence

To decide the equivalence, we look just before the `drop`, at its *site*. We form its *exposed prefix* by keeping registers hidden by earlier drops and renaming reused registers. The prefix's linear map K sends the input x of `v` to the joint state, with the dropped *block* first. Program (a) leaves `t` in $|+\rangle$ beside the untouched input, and program (b) then entangles the two:

$$K^{(a)}x = |+\rangle \otimes x, \qquad K^{(b)}x = \tfrac{1}{\sqrt{2}}\big(|0\rangle \otimes x + |1\rangle \otimes Xx\big).$$

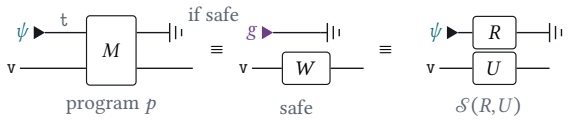
$$K^{(a)} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

($H \otimes I$; $|0\rangle \otimes I$; $K^{(a)}$; $K^{(b)} = \mathrm{CX}\,K^{(a)}$)

slices $I/\sqrt{2}, I/\sqrt{2}$ slices $I/\sqrt{2}, X/\sqrt{2}$

The matrix $|0\rangle \otimes I$ acts first, on the input x. The upper two rows of each denotation, in blue, say what the program does to `v` when `t` reads 0, and the lower two, in gold, what it does when `t` reads 1. We call these halves the *slices* of K.

A *context* is a closed program with a hole for the prefix. It supplies the input, possibly entangled with a reference, may apply gates to the returned registers, and reads every register, recording each outcome's weight. For example,




```
new v : [2] @ |0>; (t,v) = { □ }; CX t v; H t; return (t,v).
```

It undoes program (b)'s gates and reads 00 with weight 1, whereas around a witness, which leaves `t` and `v` independent, it reads 00 at most half the time. Contexts can detect any correlation, so at a safe drop the slices are all multiples of one map W on the remaining registers. Their multipliers, possibly zero, are coordinates of one fixed pure state g of the temporary, giving $Kx = g \otimes Wx$ at every input x. In program (a) both slices are equal, so g is $|+\rangle$ and W is the identity. In program (b) the slice at 1 is the slice at 0 followed by X, and no number turns the identity into X. In the picture, triangles prepare states and ground symbols drop registers.

Theorem 2.1 (without measurement). *For a well-typed, measurement-free program p, let K be its exposed map at a drop of only allocated temporaries and ψ their pure allocation state. The following are equivalent.*

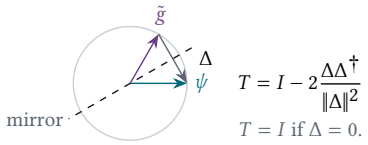
(1) $p \equiv \mathcal{S}(R,U)$ for some independent witness.

(2) $K = g \otimes W$ for a unit vector g on the block and an isometry W to all other registers.

(3) One unitary on the block alone restores ψ at every input.

Deutsch–Jozsa shows why slices need only be proportional, not equal. Its oracle O_f flips the temporary when $f(z) = 1$. Prepared in the minus state $|-\rangle$, the temporary is only negated by a flip, so the oracle leaves it in $|-\rangle$ and puts the sign $(-1)^{f(z)}$ on the data state $|z\rangle$. These signs define a data gate D_f, with $D_f|z\rangle = (-1)^{f(z)}|z\rangle$. Since $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, the upper slice is $D_f/\sqrt{2}$ and the lower is $-D_f/\sqrt{2}$. The slices are proportional for every f, and `t` ends in $|-\rangle$ at every input.

To reuse `t`, we build the return gate T from g, our *certificate*. The factorization fixes g only up to a phase, so choose $\tilde{g}$ whose overlap with ψ is real and nonnegative, moving the phase into W. The difference $\Delta = \psi - \tilde{g}$ points from the certificate to the allocation state. If $\tilde{g}$ is ψ, no gate is needed, as safe use [17, Eq. (1)] and clean safety [16, Def. 1] require of every clean ancilla. Otherwise the Householder reflection T through the pictured mirror orthogonal to Δ [14, p. 340, Eqs. (3)–(4)] sends $\tilde{g}$ to ψ on the temporary alone, leaving the rest unchanged. The witness's preparation gate R maps ψ to $\tilde{g}$. This reflection is its own inverse, so we can choose $R = T$. For any witness, undoing R gives a return gate $T = R^\dagger$, since a context must then find ψ in p as it does in the witness. For program (a), T is H, and Grover's diffusion [11, §4] is $-T$ when Δ is uniform.



3 One measurement record at a time

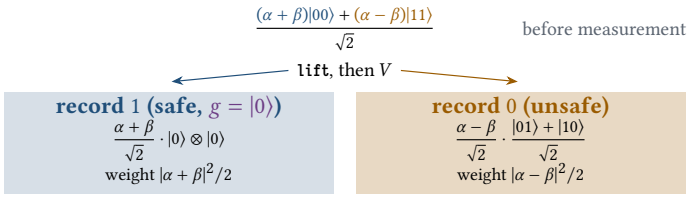
With measurement, a program can be safe on one sequence of outcomes and unsafe on another. Our `lift` measures a register and, as in dynamic lifting [10], binds an outcome that only classical `if` guards can read. In `lift[{0}] t`, outcome 1 means that `t` was read as 0, and outcome 0 means it was read as 1. A *record* o lists the outcomes along one path through the program. Its exposed map K_o is the denotation along that path, left unnormalized, so that $\|K_o x\|^2$ is the probability of the path at a normalized input x.

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>; H v; CX v t;
  k = lift[{0}] t; V t v; drop t; return v }
```

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>; new a : [2] @ |0>; H v; CX v a;
  k = lift[{0}] a; drop a; drop t; return v }
```

The picture follows program (c), with `t` first, at input $x = \alpha|0\rangle + \beta|1\rangle$. After H and the copy into `t`, the measurement keeps one term. Then V fixes $|00\rangle$ on record 1 and turns $|11\rangle$ into a Bell state on record 0. Each pictured weight is the squared magnitude of that term's coefficient.

On record 1, the slices are $W_1 = |0\rangle\langle+|$ and 0, so `t` is fixed at $|0\rangle$. W_1 gives $|0\rangle$ with weight 1 at $x = |+\rangle$, and $|0\rangle/\sqrt{2}$ with weight $1/2$ at $x = |0\rangle$. Normalizing gives $|0\rangle$ in both cases and hides this dependence on the input. Since W_1 never lengthens a vector, it is a *contraction*.

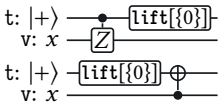


Allocating the witness's `t` and `a` in $|0\rangle$ and applying `H v; CX v a` preserve total weight, so it needs a measurement to reproduce record 1. Measuring its independent `t` would give the same probability at every input, so it measures `a` instead. Outcome 1 of `lift[{0}] a` leaves `t` and `a` in $|0\rangle$ and `v` in $W_1 x$, with record 1's weight. Thus $R = I$.

The same construction works whenever $K_o x = g \otimes W_o x$ with W_o a contraction. The helper's other outcome carries the squared length missing from $W_o x$. With fresh registers to match the input and output spaces, the two outcomes preserve norm and extend to a unitary U [24, Thm. 2]. One `lift` selects $W_o x$, while R prepares g from ψ on the block. We call record o *safe* and write $p \equiv_o \mathcal{S}(R,U)$ when no context distinguishes these selected, unnormalized maps before the drop, ignoring the fixed helper. Other records may differ.

Theorem 3.1 (on a selected record). *Let K_o be the exposed map on a record o reaching a drop of only allocated temporaries in a well-typed program p. The following are equivalent.*
(1) $p \equiv_o \mathcal{S}(R,U)$ for some independent witness using one `lift`.
(2) $K_o = g \otimes W_o$ for a unit vector g on the block and a contraction W_o to all other registers.
(3) One unitary on the block alone restores its pure allocation state ψ at every input on o.
Zero maps satisfy the three conditions, and return gates may differ between records.

Contexts cannot read records, so they see the *summed channel* [30, §2.3.2], the mixture on returned registers and a reference weighted by record probabilities. That mixture does not determine safety. At input $x = \alpha|0\rangle + \beta|1\rangle$, the programs below leave these unnormalized states on records 1 and 0 before `drop t`:



	$\|0\rangle \otimes x/\sqrt{2}$	$\|1\rangle \otimes Zx/\sqrt{2}$	safe on both
	$(\alpha\|00\rangle + \beta\|11\rangle)/\sqrt{2}$	$(\alpha\|10\rangle + \beta\|01\rangle)/\sqrt{2}$	safe on neither

Summed over records, both programs act on `v` as a standard-basis measurement, and every record has probability $1/2$ at every input, so only the per-record states tell them apart.

4 Blocks and loans

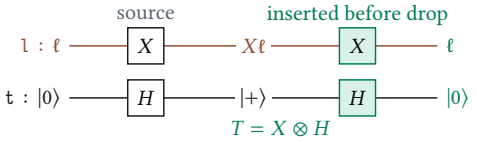
Dropping a tuple tests the named registers together. A Bell pair prepared in two fresh temporaries and left untouched is safe as a block of two, since the pair has one fixed state vector, though each half alone has the mixed state $I/2$ and so supplies no pure certificate.

We also consider blocks containing *loans*, registers borrowed in unknown states, possibly entangled with their caller, who expects them back unchanged. The witness $\mathcal{S}(R,U)$ therefore receives the loan's state ℓ, allocates the temporaries beside it (the map $J\ell = \ell \otimes \psi$, with loans first), applies R to the whole block, and lets U act only outside it. The fixed certificate g becomes an isometry B from the loan's space into the block's. The drop is then safe exactly when $K = B \otimes W$. The witness exists because a unitary R of the block carries the isometry J onto B, $RJ = B$, and the return gate $T = R^\dagger$ satisfies $TB = J$.

With measurement we again compare one record at a time, using $B_o \otimes W_o$, where B_o and the return gate may depend on the record. If the program measures nothing, or the caller's loan and reference together are uncorrelated with its other inputs, the gate restores their normalized joint state on each record that occurs. Otherwise a record can change it.

For program `lend`, with loan `l` in state ℓ, we have $B\ell = X\ell \otimes |+\rangle$, and the inserted $T = X \otimes H$ hands the unknown loan back to the caller at the drop and restores `t`.

```
prog lend (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```



5 Scope and further work

The language is linear and first order. We prove progress and preservation for its abstract machine with lifting and classical guards, and extend it to restore a safe block on each record and pool its temporaries for a later **new** of matching dimension and state. Reuse preserves the records, their weights and, on each record, the reduced state of the returned registers. Our Haskell checkers compute exactly. One builds K_0 at one-temporary sites, and the other runs the program on the basis inputs and their sum and asks that each block end in one state per record, up to phase. Both are exponential in the number of qubits, but even with only basis allocations and X, `CX` and Toffoli gates, deciding whether a drop of temporaries is safe on a record is coNP-complete (compare [17, Cor. 3.1]). Such a drop is unsafe exactly when two surviving basis inputs leave different block values.

Our OpenQASM 3 compiler uses the second check, computes return gates, and reuses basis-allocated blocks safe on every record. If program (a) ends with **new** `y : [2] @ |1>;` **return** `(v,y)` instead, its circuit uses two qubits where fresh allocation uses three: the return gate H restores `t`, and X prepares `y` on `t`'s wire. Measuring `t` and resetting it to $|0\rangle$, as qubit-reuse compilers do [8, 15], would save the qubit too, but a reset destroys a loan's state, which the return gate hands back. Only basis allocations compile, program (c)'s V is unsupported, and no loan may be dropped after a **lift** or an **if**. Return gates are numerical, and for a general loan return, $TB = J$ is checked entrywise, to within 10^{-9}.

Testing the state a temporary ends in, not the gates that produced it, our criterion accepts the balanced Deutsch–Jozsa drop that Silq and Unqomp refuse and Qutes does not reclaim (neither dead nor output-isolable [9, Defs. 2–6]). Qurts puts lifetimes in types [13, §3] and Reqomp trades qubits for gates [21, §4], and both reclaim by uncomputing. ReQWIRE verifies that an ancilla is restored [23, §4], as Q# [19] and fractional types [7] require. Local rotations repair separable errors on a dirty ancilla [16, §5.2, Alg. 2], where our one gate per record restores a whole block. On measurement-free circuits with $|0\rangle$ ancillas, the validity test of slices summing to a unitary [17, Def. 3.1] is incomparable with safety, as program (a)'s slices sum to $\sqrt{2}I$, yet its drop is safe, and those of **new** `t : [2] @ |0>; CX v t;` **drop** `t` sum to I, yet its drop is unsafe. QBorrow needs the loan back untouched after each denoted operation [28, Def. 5.1] and Bona after each borrowing body [31, Def. 3.2], and separation logics prove its return by hand [18, 29]. Our return gate undoes any isometry B.

We did not consider loops or quantum recursion. What certifies a temporary dropped after a loop, possibly in superposition, is open. We also took every unitary to be a gate, but a fixed gate set may lack the return gate. Circuits of X, `CX`, Toffoli and $H \otimes H$ have dyadic entries [2, Cor. 5.6]. After $H \otimes H$ on `t` and `v` the drop of `t` is safe, yet no such circuit on `t` alone restores it, since that needs the non-dyadic entries $\pm 1/\sqrt{2}$.

References

[1] Samson Abramsky and Bob Coecke. 2004. A Categorical Semantics of Quantum Protocols. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS 2004)*. 415–425. doi:10.1109/LICS.2004.1319636

[2] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum* 4, Article 252 (2020). doi:10.22331/q-2020-04-06-252

[3] Charles H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532. doi:10.1147/rd.176.0525

[4] Charles H. Bennett. 1989. Time/Space Trade-Offs for Reversible Computation. *SIAM J. Comput.* 18, 4 (1989), 766–776. doi:10.1137/0218053

[5] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*. ACM, 286–300. doi:10.1145/3385412.3386007

[6] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. How to Bake a Quantum Π. *Proceedings of the ACM on Programming Languages* 8, ICFP (2024), 1–29. doi:10.1145/3674625

[7] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[8] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2023. Qubit-Reuse Compilation with Mid-Circuit Measurement and Reset. *Physical Review X* 13, 4 (2023), 041057. doi:10.1103/PhysRevX.13.041057

[9] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph] (numbered as in version 1).

[10] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (2023), 26 pages. doi:10.1145/3571204

[11] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[12] Chris Heunen and Robin Kaarsgaard. 2022. Quantum Information Effects. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 2 (2022). doi:10.1145/3498663

[13] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 28 pages. doi:10.1145/3704842

[14] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[15] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)*. 59–71. doi:10.1145/3582016.3582030

[16] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 (numbered as in arXiv:2608.13099v1).

[17] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. *Proceedings of the ACM on Programming Languages* 10, OOPSLA2 (2026), 2270–2298. doi:10.1145/3839528 (numbered as in arXiv:2608.09578v2).

[18] Yusuke Matsushita, Kengo Hirata, Ryo Wakizaka, and Emanuele D'Osualdo. 2026. RapunSL: Untangling Quantum Computing with Separation, Linear Combination and Mixing. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 6 (2026). doi:10.1145/3776648

[19] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[20] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[21] Anouk Paradis, Benjamin Bichsel, and Martin Vechev. 2024. Reqomp: Space-Constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. doi:10.22331/q-2024-02-19-1258

[22] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. 846–858. doi:10.1145/3009837.3009894

[23] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[24] P. L. Robinson. 2018. Julia Operators and Halmos Dilations. arXiv:1803.09329 doi:10.48550/arXiv.1803.09329

[25] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Computer Science* 14, 4 (2004), 527–586. doi:10.1017/S0960129504004256

[26] Peter Selinger and Benoît Valiron. 2006. A Lambda Calculus for Quantum Computation with Classical Control. *Mathematical Structures in Computer Science* 16, 3 (2006), 527–552. doi:10.1017/S0960129506005238

[27] Sam Staton. 2015. Algebraic Effects, Linearity, and Quantum Programming Languages. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '15)*. 395–406. doi:10.1145/2676726.2676999

[28] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. Borrowing Dirty Qubits in Quantum Programs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. 274–289. doi:10.1145/3779212.3790134 (numbered as in arXiv:2508.17190v1).

[29] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. An Interpretation of Bunched Logic for Reasoning about Heap-Manipulating Quantum Programs. arXiv:2409.10153 [cs.PL] v2, 5 Aug 2026; v1 (16 Sep 2024) was titled "BI-based Reasoning about Quantum Programs with Heap Manipulations".

[30] John Watrous. 2018. *The Theory of Quantum Information*. Cambridge University Press. doi:10.1017/9781316848142 (numbered as in the 2018 prepublication draft).

[31] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL] (numbered as in version 1).